

What canvas is

- **Definition:** The HTML `<canvas>` element provides a drawable 2D bitmap you control via JavaScript's Canvas API.
- **Coordinate system:** Origin at the top-left; x increases to the right, y increases downward.
- **Immediate mode:** Canvas draws pixels; it doesn't retain objects. You must re-draw everything for changes or animation.

Basic markup and context

```
<canvas id="board" width="600" height="400"></canvas>
<script>
  // 1) Get the canvas element
  const canvas = document.getElementById('board');

  // 2) Get the 2D drawing context
  const ctx = canvas.getContext('2d');
</script>
```

- **Width vs CSS:** Set `width` / `height` attributes for actual pixel resolution. CSS size scales visually and can blur.
- **Fallback content:** Put text inside `<canvas>...</canvas>` to show if canvas isn't supported.

```
<canvas id="board" width="600" height="400">
  Your browser doesn't support canvas.
</canvas>
```

Drawing basics: rectangles, lines, paths

Rectangles (fastest primitives)

```
<canvas id="c1" width="300" height="150"></canvas>
<script>
const ctx = document.getElementById('c1').getContext('2d');

// Filled rectangle
ctx.fillStyle = '#3498db';           // 1) Set fill color
ctx.fillRect(20, 20, 100, 60);       // 2) Draw filled rect

// Stroked rectangle
ctx.strokeStyle = '#2c3e50';         // 3) Set stroke color
ctx.lineWidth = 4;                   // 4) Set stroke width
ctx.strokeRect(150, 20, 100, 60);    // 5) Outline rect

// Clear a region (makes it transparent)
ctx.clearRect(30, 30, 30, 30);        // 6) Clear part of the first rect
</script>
```

Lines and paths (beginPath, moveTo,.lineTo)

```
<canvas id="c2" width="300" height="150"></canvas>
<script>
const ctx = document.getElementById('c2').getContext('2d');

ctx.beginPath();           // 1) Start a new path
ctx.moveTo(20, 20);        // 2) Move pen to start
ctx.lineTo(280, 20);       // 3) Draw a line
ctx.lineTo(280, 130);      // 4) Another line
ctx.closePath();           // 5) Close path back to start

ctx.strokeStyle = '#e67e22'; // 6) Outline color
ctx.lineWidth = 6;         // 7) Thickness
ctx.stroke();              // 8) Render outline

ctx.fillStyle = 'rgba(230, 126, 34, 0.2)'; // 9) Fill color w/ alpha
ctx.fill();                // 10) Fill interior
</script>
```

Arcs and circles

```
<canvas id="c3" width="300" height="150"></canvas>
<script>
const ctx = document.getElementById('c3').getContext('2d');

ctx.beginPath();
ctx.arc(150, 75, 50, 0, Math.PI * 2); // 1) Center (150,75), radius 50, full circle
ctx.fillStyle = '#9b59b6';
ctx.fill();

ctx.beginPath();
ctx.arc(150, 75, 60, 0, Math.PI, false); // 2) Half circle
ctx.strokeStyle = '#8e44ad';
ctx.stroke();
</script>
```

Styling and text

Fill, stroke, and line styles

- **fillStyle**: Color, gradient, or pattern for shapes' interior.
- **strokeStyle**: Color, gradient, or pattern for outlines.
- **lineWidth**, **lineJoin**, **lineCap**: Control stroke thickness and joins/caps.

```
<canvas id="c4" width="320" height="160"></canvas>
<script>
const ctx = document.getElementById('c4').getContext('2d');

ctx.lineWidth = 12;
ctx.lineJoin = 'round';           // 'miter' | 'round' | 'bevel'
ctx.lineCap = 'round';           // 'butt' | 'round' | 'square'
ctx.strokeStyle = '#16a085';

ctx.beginPath();
ctx.moveTo(20, 40);
ctx.lineTo(160, 40);
ctx.lineTo(160, 120);
ctx.stroke();
```

```
ctx.fillStyle = '#1abc9c';
ctx.fillRect(200, 40, 100, 60);
</script>
```

Text drawing

```
<canvas id="c5" width="400" height="160"></canvas>
<script>
const ctx = document.getElementById('c5').getContext('2d');

// 1) Set font (CSS-like)
ctx.font = 'bold 24px system-ui';

// 2) Text alignment & baseline
ctx.textAlign = 'center';      // 'left' | 'center' | 'right'
ctx.textBaseline = 'middle';   // 'top' | 'alphabetic' | 'bottom' | etc.

// 3) Fill and stroke text
ctx.fillStyle = '#2c3e50';
ctx.fillText('Hello Canvas', 200, 80);

ctx.strokeStyle = '#e74c3c';
ctx.lineWidth = 2;
ctx.strokeText('Hello Canvas', 200, 80);

// 4) Measure text width
const metrics = ctx.measureText('Hello Canvas');
console.log(metrics.width);
</script>
```

Images, patterns, and gradients

Drawing images

```
<canvas id="c6" width="400" height="200"></canvas>
<script>
const ctx = document.getElementById('c6').getContext('2d');
const img = new Image();
```

```

img.src = 'photo.jpg'; // Same-origin or CORS-enabled URL

img.onload = () => {
  // 1) Draw as-is at (20,20)
  ctx.drawImage(img, 20, 20);

  // 2) Draw scaled
  ctx.drawImage(img, 200, 20, 160, 120);

  // 3) Draw cropped: sourceRect -> destRect
  // drawImage(image, sx, sy, sWidth, sHeight, dx, dy, dWidth, dHeight)
  ctx.drawImage(img, 50, 50, 100, 80, 20, 120, 140, 100);
};
</script>

```

Gradients and patterns

```

<canvas id="c7" width="400" height="200"></canvas>
<script>
const ctx = document.getElementById('c7').getContext('2d');

// Linear gradient
const grad = ctx.createLinearGradient(0, 0, 400, 0);
grad.addColorStop(0, '#3498db');
grad.addColorStop(1, '#2ecc71');
ctx.fillStyle = grad;
ctx.fillRect(0, 0, 400, 80);

// Pattern (tile an image)
const img = new Image();
img.src = 'tile.png';
img.onload = () => {
  const pattern = ctx.createPattern(img, 'repeat'); // 'repeat' | 'repeat-x' | 'repeat-y'
  ctx.fillStyle = pattern;
  ctx.fillRect(0, 100, 400, 100);
};
</script>

```

Transforms, state, and composition

Save/restore and transforms

```
<canvas id="c8" width="400" height="200"></canvas>
<script>
const ctx = document.getElementById('c8').getContext('2d');

// Save initial state
ctx.save();

ctx.translate(200, 100);      // Move origin
ctx.rotate(Math.PI / 6);     // Rotate 30 degrees
ctx.scale(1.2, 1.2);         // Scale x and y

ctx.fillStyle = '#f1c40f';
ctx.fillRect(-50, -30, 100, 60); // Draw around new origin

// Restore to original
ctx.restore();

ctx.strokeStyle = '#34495e';
ctx.strokeRect(10, 10, 80, 40); // Untransformed
</script>
```

Global alpha and composite operations

```
<canvas id="c9" width="320" height="160"></canvas>
<script>
const ctx = document.getElementById('c9').getContext('2d');

ctx.fillStyle = '#e74c3c';
ctx.fillRect(40, 40, 80, 80);

ctx.globalAlpha = 0.5;                // Make subsequent draws semi-transparent
ctx.fillStyle = '#3498db';
ctx.fillRect(80, 80, 80, 80);
ctx.globalAlpha = 1.0;                // Reset

ctx.globalCompositeOperation = 'multiply'; // Blend mode (e.g., 'source-over', 'destination-atop')
ctx.fillStyle = '#2ecc71';
ctx.fillRect(120, 40, 80, 80);
</script>
```

Animation and interaction

Time-based animation loop

```
<canvas id="c10" width="600" height="200"></canvas>
<script>
const ctx = document.getElementById('c10').getContext('2d');
let x = 0;
let last = performance.now();

function frame(now) {
  const dt = (now - last) / 1000;    // 1) Seconds since last frame
  last = now;

  // 2) Update state time-based (speed = 100 px/s)
  x += 100 * dt;
  if (x > ctx.canvas.width + 50) x = -50;

  // 3) Clear canvas
  ctx.clearRect(0, 0, ctx.canvas.width, ctx.canvas.height);

  // 4) Draw
  ctx.fillStyle = '#1abc9c';
  ctx.fillRect(x, 80, 50, 50);

  requestAnimationFrame(frame);
}
requestAnimationFrame(frame);
</script>
```

Mouse interaction and hit detection

```
<canvas id="c11" width="400" height="200" style="border:1px solid #ccc"></canvas>
<script>
const canvas = document.getElementById('c11');
const ctx = canvas.getContext('2d');

const rect = { x: 50, y: 50, w: 100, h: 80 };
let hover = false;
```

```

// Draw function
function draw() {
  ctx.clearRect(0, 0, canvas.width, canvas.height);
  ctx.fillStyle = hover ? '#e67e22' : '#3498db';
  ctx.fillRect(rect.x, rect.y, rect.w, rect.h);
}

// Convert client coords to canvas coords (account for CSS scaling)
function getCanvasPoint(e) {
  const r = canvas.getBoundingClientRect();
  const scaleX = canvas.width / r.width;
  const scaleY = canvas.height / r.height;
  return {
    x: (e.clientX - r.left) * scaleX,
    y: (e.clientY - r.top) * scaleY
  };
}

canvas.addEventListener('mousemove', (e) => {
  const p = getCanvasPoint(e);
  hover = p.x >= rect.x && p.x <= rect.x + rect.w &&
    p.y >= rect.y && p.y <= rect.y + rect.h;
  draw();
});

draw();
</script>

```

Pixel manipulation and effects

Get image data and alter pixels

```

<canvas id="c12" width="320" height="180"></canvas>
<script>
const ctx = document.getElementById('c12').getContext('2d');

// 1) Draw a gradient background
const g = ctx.createLinearGradient(0, 0, 320, 0);
g.addColorStop(0, '#ff7675');
g.addColorStop(1, '#74b9ff');
ctx.fillStyle = g;
ctx.fillRect(0, 0, 320, 180);

// 2) Read pixels

```

```

const imgData = ctx.getImageData(0, 0, 320, 180);
const data = imgData.data; // Uint8ClampedArray: [r,g,b,a, r,g,b,a, ...]

// 3) Convert to grayscale
for (let i = 0; i < data.length; i += 4) {
  const r = data[i], g = data[i+1], b = data[i+2];
  const gray = (r + g + b) / 3;
  data[i] = data[i+1] = data[i+2] = gray;
}

// 4) Write pixels back
ctx.putImageData(imgData, 0, 0);
</script>

```

- **Performance tip:** Use small regions in `getImageData` / `putImageData` when possible; avoid full-canvas operations per frame.

High-DPI scaling, resizing, and saving

Handle device pixel ratio

```

<canvas id="c13" style="width:600px; height:300px;"></canvas>
<script>
const canvas = document.getElementById('c13');
const ctx = canvas.getContext('2d');

function resize() {
  const dpr = window.devicePixelRatio || 1;
  const rect = canvas.getBoundingClientRect();

  // 1) Set actual pixel size
  canvas.width = Math.round(rect.width * dpr);
  canvas.height = Math.round(rect.height * dpr);

  // 2) Scale context so drawing uses CSS pixels
  ctx.setTransform(dpr, 0, 0, dpr, 0, 0);

  // 3) Redraw
  ctx.clearRect(0, 0, canvas.width, canvas.height);
  ctx.fillStyle = '#2ecc71';
  ctx.fillRect(20, 20, 200, 100);
}

window.addEventListener('resize', resize);

```

```
resize();  
</script>
```

Export as image

```
<button id="saveBtn">Save PNG</button>  
<canvas id="c14" width="300" height="150"></canvas>  
<script>  
  const canvas = document.getElementById('c14');  
  const ctx = canvas.getContext('2d');  
  
  // Draw something  
  ctx.fillStyle = '#f39c12';  
  ctx.fillRect(40, 40, 100, 60);  
  
  document.getElementById('saveBtn').addEventListener('click', () => {  
    const url = canvas.toDataURL('image/png'); // 'image/jpeg' optional  
    const a = document.createElement('a');  
    a.href = url;  
    a.download = 'canvas.png';  
    a.click();  
  });  
</script>
```

Real-world mini projects

Progress ring (animated, accessible label)

```
<div aria-live="polite" id="label">Loading... 0%</div>  
<canvas id="ring" width="200" height="200"></canvas>  
<script>  
  const canvas = document.getElementById('ring');  
  const ctx = canvas.getContext('2d');  
  const label = document.getElementById('label');  
  let progress = 0;  
  
  function draw() {  
    const { width, height } = canvas;  
    const cx = width / 2, cy = height / 2, r = 80;
```

```

ctx.clearRect(0, 0, width, height);

// Track
ctx.beginPath();
ctx.arc(cx, cy, r, 0, Math.PI * 2);
ctx.strokeStyle = '#ddd';
ctx.lineWidth = 12;
ctx.stroke();

// Progress arc
ctx.beginPath();
const end = (progress / 100) * Math.PI * 2 - Math.PI / 2; // start at top
ctx.arc(cx, cy, r, -Math.PI / 2, end);
ctx.strokeStyle = '#27ae60';
ctx.lineWidth = 12;
ctx.lineCap = 'round';
ctx.stroke();

// Text
ctx.font = 'bold 24px system-ui';
ctx.textAlign = 'center';
ctx.textBaseline = 'middle';
ctx.fillStyle = '#2c3e50';
ctx.fillText(`${progress}%`, cx, cy);

label.textContent = `Loading... ${progress}%`; // Accessible live region
}

function animate() {
  progress = Math.min(100, progress + 1);
  draw();
  if (progress < 100) requestAnimationFrame(animate);
}
draw();
animate();
</script>

```

Signature pad (pointer events, pressure-aware)

```

<canvas id="pad" width="600" height="200" style="border:1px solid #aaa"></canvas>
<button id="clear">Clear</button>
<script>
const canvas = document.getElementById('pad');
const ctx = canvas.getContext('2d');

```

```

let drawing = false;
let last = null;

function getPoint(e) {
  const r = canvas.getBoundingClientRect();
  const sx = canvas.width / r.width;
  const sy = canvas.height / r.height;
  return { x: (e.clientX - r.left) * sx, y: (e.clientY - r.top) * sy, p: e.pressure || 1 }
}

canvas.addEventListener('pointerdown', (e) => {
  drawing = true;
  last = getPoint(e);
});

canvas.addEventListener('pointermove', (e) => {
  if (!drawing) return;
  const cur = getPoint(e);
  ctx.lineCap = 'round';
  ctx.lineJoin = 'round';
  ctx.strokeStyle = '#000';
  ctx.lineWidth = 1 + cur.p * 3; // Vary width by pressure
  ctx.beginPath();
  ctx.moveTo(last.x, last.y);
  ctx.lineTo(cur.x, cur.y);
  ctx.stroke();
  last = cur;
});

canvas.addEventListener('pointerup', () => drawing = false);
canvas.addEventListener('pointerleave', () => drawing = false);

document.getElementById('clear').addEventListener('click', () => {
  ctx.clearRect(0, 0, canvas.width, canvas.height);
});
</script>

```

Common pitfalls and best practices

- **Canvas size:** Always set `width` / `height` attributes to control pixel resolution. CSS-only sizing leads to blurriness.
- **Redraw model:** Canvas doesn't store shapes. For interaction or animation, maintain your own state and redraw each frame.
- **Coordinate conversion:** Use `getBoundingClientRect()` to convert pointer positions correctly, especially when CSS scaling differs from pixel size.

Quick reference (most-used APIs)

- **Context:** `const ctx = canvas.getContext('2d')`
- **Rectangles:** `fillRect(x,y,w,h)`, `strokeRect(x,y,w,h)`, `clearRect(x,y,w,h)`
- **Paths:** `beginPath()`, `moveTo(x,y)`, `lineTo(x,y)`, `arc(x,y,r,sa,ea)`, `closePath()`, `stroke()`, `fill()`
- **Styles:** `fillStyle`, `strokeStyle`, `lineWidth`, `lineCap`, `lineJoin`, `globalAlpha`
- **Text:** `font`, `textAlign`, `textBaseline`, `fillText(text,x,y)`, `strokeText(text,x,y)`, `measureText(text)`
- **Images:** `drawImage(img, ...)`, `createPattern(img, repeat)`, `createLinearGradient(...)`, `createRadialGradient(...)`
- **Transforms:** `save()`, `restore()`, `translate(x,y)`, `rotate(rad)`, `scale(x,y)`, `setTransform(a,b,c,d,e,f)`
- **Pixels:** `getImageData(x,y,w,h)`, `putImageData(imgData,x,y)`
- **Composition:** `globalCompositeOperation = 'source-over'` (default) and many others
- **Export:** `toDataURL(type)`